

Partie 8.2 — Serveur Web sécurisé (HTTPS) + PHP + MariaDB + phpMyAdmin

Objectifs de la séance

À l'issue de cette séance, vous serez capables de :

- Comprendre le fonctionnement du protocole HTTPS
- Sécuriser un serveur Apache avec TLS/SSL
- Installer et configurer PHP
- Installer et sécuriser MariaDB
- Connecter PHP à une base de données
- Installer et sécuriser phpMyAdmin
- Comprendre les interactions d'une architecture **LAMP sécurisée**

Etape 0 : Comprendre HTTPS

Le problème : HTTP (HyperText Transfer Protocol) Port 80 (Pas de cadenas, pas de sécurité).

Le HTTP "classique" fonctionne en **clair**. Imaginez que chaque donnée que vous envoyez (un login, un mot de passe, un numéro de CB) est écrite sur une feuille de papier que n'importe qui peut lire sur le trajet entre votre PC et le serveur.

- **Le risque** : L'attaque de type **Man-in-the-Middle (MITM)**. Un pirate sur le même réseau Wi-Fi que vous peut intercepter et lire vos données sans aucun effort.

La solution : HTTPS (HyperText Transfer Protocol Secure) Port 443 (Cadenas vert, données chiffrées via TLS).

Le HTTPS n'est pas un protocole totalement différent ; c'est du HTTP "encapsulé" dans une couche de sécurité appelée **TLS** (Transport Layer Security).

Note : On parle souvent de SSL, mais c'est l'ancien nom. Aujourd'hui, on utilise techniquement le TLS.

Comment ça marche ?

Le navigateur et le serveur s'entendent sur une "clé" secrète. Toutes les données sont mélangées (chiffrées) avant de partir sur Internet. Si un attaquant intercepte le message, il ne verra qu'une suite de caractères incompréhensibles :

- **En HTTP** : password=1234
- **En HTTPS** : ax8#9z!Lp2@...

Les trois piliers de HTTPS

Pour l'examen, reprenez bien ces trois concepts clés. HTTPS ne sert pas qu'à "cacher" les infos, il apporte :

1. **Confidentialité** : Seul le destinataire peut lire l'information. Même si quelqu'un "écoute" le réseau, il ne comprend rien.
2. **Intégrité** : On garantit que les données n'ont pas été modifiées pendant le voyage. Si un pirate change un seul bit, la signature numérique sera invalide et la connexion sera coupée.
3. **Authentification** : C'est le rôle du **Certificat SSL/TLS**. Il prouve que vous êtes bien sur le site officiel (ex: google.com) et non sur une copie pirate créée par un hacker.

Etape 1 : Activation du module SSL Apache

Apache utilise un module spécifique pour HTTPS.

Activer SSL

```
sudo a2enmod ssl
```

Active les fonctionnalités TLS dans Apache.

```
sudo systemctl restart apache2
```

Redémarre Apache

Etape 2 : Création d'un certificat SSL local

Dans un environnement réel, on utilise **Let's Encrypt**.
Ici, nous créons un certificat **auto-signé** pour les tests.

Créer le certificat

Quand vous lancerez la commande openssl, le terminal vous posera une série de questions (Pays, Ville, Organisation).

- Le plus important : Le Common Name (CN). Mettez votre_login.fr.
- La sécurité : Notez que la clé privée est stockée dans /etc/ssl/private/. Ce dossier est normalement protégé pour que seul l'utilisateur root puisse y accéder. C'est crucial pour la sécurité de votre serveur.

```
sudo openssl req
-x509
-nodes -days 365 \
-newkey rsa:2048 \
-keyout /etc/ssl/private/apache-selfsigned.key \
-out /etc/ssl/certs/apache-selfsigned.crt
```

Explication

Option	Rôle
-x509	certificat auto-signé
-nodes	pas de mot de passe
rsa:2048	clé sécurisée
365	durée de validité

Lors de la saisie :

Common Name = votre_login.fr

Etape 3 : Création du VirtualHost HTTPS

Créer un nouveau fichier :

```
sudo nano /etc/apache2/sites-available/votre_login-ssl.conf
```

Configuration HTTPS

```
<VirtualHost *:443>

    ServerName votre_login.fr
    DocumentRoot /var/www/mon_serveur

    SSLEngine on
    SSLCertificateFile /etc/ssl/certs/apache-selfsigned.crt
    SSLCertificateKeyFile /etc/ssl/private/apache-selfsigned.key

    <Directory /var/www/mon_serveur>
        Options FollowSymLinks
        AllowOverride None
        Require all granted
    </Directory>

</VirtualHost>
```

Explication

Directive	Rôle
443	port HTTPS
SSLEngine	active TLS
CertificateFile	certificat

Directive	Rôle
KeyFile	clé privée

Activer le site :

`sudo a2ensite votre_login-ssl.conf`

Tester la configuration :

`sudo apache2ctl configtest`

Redémarrer Apache :

`sudo systemctl restart apache2`

Ouvrir :

`https://votre_login.fr`

Avertissement navigateur normal (certificat local).



Redirection du 80 au 443 HTTPS

`<VirtualHost *:80>`

`ServerName votre_login.fr`

`ServerAlias votre-login.fr`

`# Redirection permanente (301) vers le HTTPS`

`Redirect permanent / https://www.votre-login.fr/`

`</VirtualHost>`

Ouvrir un navigateur :

`http://votre_login.fr`

Etape 4 : Installation de PHP

Rôle de PHP

1. Le concept de "Page Dynamique"

Contrairement au HTML pur qui est fixe (comme une affiche sur un mur), PHP agit comme un **générateur de code**.

- **Page statique (.html)** : Le serveur envoie le fichier tel quel. Tous les visiteurs voient exactement la même chose.
- **Page dynamique (.php)** : Le serveur "fabrique" la page sur mesure au moment où l'utilisateur la demande (en fonction de l'heure, de l'utilisateur connecté, ou des données en base de données).

2. Le schéma de fonctionnement (Le cycle Requête/Réponse)

C'est ici que l'interaction entre le navigateur, le serveur Web (Apache) et l'interpréteur PHP prend tout son sens.

1. **Navigateur (Client)** : L'utilisateur tape une URL ou clique sur un lien (ex: profil.php). Il envoie une **requête HTTP**.
2. **Apache (Serveur Web)** : Il reçoit la requête. S'il voit l'extension .php, il comprend qu'il ne peut pas lire ce fichier seul. Il passe le relais à **PHP**.
3. **PHP (Moteur)** : Il exécute le code. Il peut aller chercher des informations dans une base de données (MySQL), faire des calculs ou vérifier des mots de passe.
4. **Résultat HTML** : Une fois son travail fini, PHP produit du texte au format **HTML**.
5. **Retour au client** : Apache renvoie ce HTML tout frais au navigateur, qui l'affiche.

Le saviez-vous ? Le navigateur ne voit **jamais** le code PHP (les balises <?php ... ?>). Il ne reçoit que le résultat final, le HTML. C'est pour cela que PHP est un langage dit "**Server-Side**" (côté serveur).

3. Apport du moteur PHP

Dans vos futurs projets (comme les PPE ou les ateliers de professionnalisation), PHP vous servira à :

- **Gérer des formulaires** : Récupérer les données d'une inscription ou d'un contact.
- **Sécuriser l'accès** : Créer des sessions pour que seuls les administrateurs voient certaines pages.
- **Interagir avec une base de données** : Afficher un catalogue de produits stockés dans MariaDB ou MySQL.

Installation

```
sudo apt install php libapache2-mod-php php-mysql
```

Redémarrer Apache :

```
sudo systemctl restart apache2
```

Vérifier PHP

Créer :

```
nano ~/mon_serveur/info.php
```

Contenu :

```
<?php  
phpinfo();  
?>
```

Vérifie que le chemin /var/www/mon_serveur existe bien et que les permissions permettent la lecture à l'utilisateur www-data

```
sudo chown -R www-data:www-data /var/www/mon_serveur
```

Tester :

```
https://votre_login.fr/info.php
```

Page d'information PHP affichée.

Etape 5 : Installation de MariaDB (serveur de base de données)

1. Pourquoi MariaDB plutôt que MySQL ?

Il faut d'abord clarifier la notion de "**Fork**". Après le rachat de MySQL par Oracle, les créateurs originaux ont lancé **MariaDB** pour garantir que le moteur reste **100% libre (Open Source)**.

- **Compatibilité** : C'est un remplacement "goutte pour goutte" (*drop-in replacement*). Une commande qui marche sous MySQL marchera sous MariaDB.
- **Performance** : MariaDB est souvent jugé plus rapide et plus moderne dans sa gestion de la mémoire.
- **Éthique** : En SIO, on privilégie souvent les solutions communautaires et libres.

2. Les 3 piliers de MariaDB en développement

A. Le Stockage de Données (La Mémoire du site)

Sans base de données, un site "oublie" tout dès qu'on rafraîchit la page. MariaDB permet de structurer les informations dans des **tables** (colonnes et lignes).

- *Exemple* : Une table Produits, une table Commandes.

B. La Gestion des Comptes Utilisateurs

MariaDB ne stocke pas seulement les données des clients, il gère aussi **qui a le droit de faire quoi** sur le serveur de base de données.

- **Sécurité** : On crée des utilisateurs SQL spécifiques (ex: user_web) qui n'ont le droit que de lire ou d'écrire, mais pas de supprimer toute la base.

C. Le moteur des Applications Web Dynamiques

C'est la pièce manquante du puzzle Apache/PHP. MariaDB est le "**cerveau**" où PHP va piocher ses informations.

3. Le rôle de MariaDB dans le flux de données

Pour vos étudiants, vous pouvez résumer le parcours d'une information ainsi :

1. **Le Client** demande la liste des produits (via le navigateur).
2. **Apache** passe la main à **PHP**.
3. **PHP** envoie une requête **SQL** à **MariaDB** ("*Donne-moi tous les produits en stock*").
4. **MariaDB** répond à PHP avec les données.
5. **PHP** transforme ces données en tableau HTML propre.
6. **Apache** renvoie le tout au client.

C'est ce qu'on appelle l'architecture **3-tiers** (Client / Serveur Web / Serveur de données).

Installation

```
sudo apt install mariadb-server
```

Vérifier le service :

```
systemctl status mariadb
```

Etape 6 : Sécurisation de MariaDB

Lancer l'outil de sécurisation :

```
sudo mysql_secure_installation
```

Réponses conseillées :

Question	Réponse
Set root password	YES
Remove anonymous users	YES
Disallow remote login	YES
Remove test database	YES
Reload privilege tables	YES

Explication

Supprime les accès dangereux par défaut.

Création d'une base de données

Connexion :

```
sudo mysql
```

Créer une base :

```
CREATE DATABASE testweb;
```

Créer un utilisateur :

```
CREATE USER 'webuser'@'localhost' IDENTIFIED BY 'Password123!';  
GRANT ALL PRIVILEGES ON testweb.* TO 'webuser'@'localhost';  
FLUSH PRIVILEGES;  
EXIT;
```

Etape 7 : Test PHP + MariaDB

Créer :

```
nano ~/mon_serveur/testdb.php
<?php
$conn = new mysqli("localhost","webuser","Password123!","testweb");

if ($conn->connect_error) {
    die("Connexion échouée");
}

echo "Connexion réussie à MariaDB !";
?>
```

Tester :

https://votre_login.fr/testdb.php

Etape 8 : Installation de phpMyAdmin

1. phpMyAdmin

phpMyAdmin n'est pas une base de données. C'est une **interface web** (écrite en PHP, d'où son nom) qui permet d'administrer MariaDB sans taper une seule ligne de code SQL.

- **Sans phpMyAdmin** : On gère la base dans un terminal noir et blanc, avec des commandes complexes (SELECT * FROM...).
- **Avec phpMyAdmin** : On gère la base avec sa souris, des boutons, des onglets et des formulaires.

2. Son rôle technique (Le "Couteau Suisse")

Pour un futur technicien SIO, phpMyAdmin sert à quatre choses essentielles :

1. **Visualiser** : Voir le contenu des tables sous forme de tableaux (comme un Excel).
2. **Structurer** : Créer des bases, ajouter des colonnes ou modifier des types de données (INT, VARCHAR, DATE) en quelques clics.
3. **Importer / Exporter** : Sauvegarder une base de données dans un fichier .sql ou restaurer une base existante (indispensable pour les examens de BTS !).
4. **Tester** : Écrire ses requêtes SQL manuellement pour vérifier si elles fonctionnent avant de les copier-coller dans le code PHP.

3. Comment ça marche ? (Le flux)

Il est important que les étudiants ne confondent pas l'outil et la donnée.

Voici le chemin parcouru :

Navigateur (Utilisateur) -> **Apache** -> **PHP** (Exécute phpMyAdmin) -> **MariaDB** (Récupère les données).

En résumé, **phpMyAdmin est un site web dont le seul but est de parler à votre base de données.**

Installation

```
sudo apt install phpmyadmin
```

Choisir :

- apache2
- configurer base avec dbconfig-common

Activer l'intégration Apache :

```
sudo ln -s /usr/share/phpmyadmin /var/www/html/phpmyadmin
```

Redémarrer Apache :

sudo systemctl restart apache2

Accès

<https://localhost/phpmyadmin> ou <https://votre-login.fr/phpmyadmin>

Connexion avec :

- utilisateur : webuser
- mot de passe défini précédemment

Etape 9 : Sécurisation minimale phpMyAdmin (important)

Éditer :

sudo nano /etc/apache2/conf-available/phpmyadmin.conf

Ajouter :

Require local

accès uniquement depuis la machine locale.

Recharger Apache :

sudo systemctl reload apache2

Si Apache refuse de redémarrer, utilise toujours cette commande pour voir l'erreur précise :

journalctl -xeu apache2.service

Vérifications finales

Vous devez pouvoir accéder à :

- https://votre_login.fr
- https://votre_login.fr/info.php
- https://votre_login.fr/testdb.php
- <https://localhost/phpmyadmin>

Architecture obtenue

Composant	Rôle	Métaphore
Apache	Serveur Web	Le serveur au restaurant (il prend la commande).
PHP	Langage de script	Le cuisinier (il prépare le plat/la page).
MariaDB	Base de données	Le garde-manger (où sont les ingrédients/infos).
phpMyAdmin	Interface de gestion	L'inventaire du garde-manger sur tablette tactile.

Stack **LAMP sécurisée** fonctionnelle.

Compétences acquises

Vous savez maintenant :

- sécuriser un serveur Web en HTTPS
- utiliser PHP avec Apache
- installer une base MariaDB
- créer utilisateurs et bases
- connecter PHP à une base
- administrer via phpMyAdmin
- comprendre une architecture Web complète